

134. Passer à la salle suivante, sans bouton

Ce n'est pas un bouton qui change de salle : c'est le personnage qui sort du cadre.

1. Ce qu'il faut comprendre

La leçon « Passer d'un écran à l'autre » changeait de scène **sur un appui**. Ici, rien de tel : c'est la POSITION du héros qui décide — sortir par la droite fait apparaître la salle suivante, avec le héros collé au bord gauche.

La même règle que `allerA()` s'applique : `ecran(0)`, `effacer()`, redessiner, puis `ecran(1)` — toujours dans cet ordre, pour ne pas voir l'ancienne salle se mélanger à la nouvelle pendant qu'on l'efface.

Le héros ne s'arrête jamais à un bord comme dans « Le garder dans l'écran » : ici, dépasser le bord est justement ce qui déclenche le changement. Les deux techniques répondent à deux besoins opposés, avec le même test `x > DROITE_MAX`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **631 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	60 o	← la console, à l'allumage — la mise en route de la console
dessinerSalle()	68 o	← main() — ta fonction
main()	148 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	67 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
EffacerFond	17 o	← effacer() — vide tout le fond
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	96 o	← CopierTuiles — 5 lettres sur 43, et 1 tuile dessinée
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins
"SALLE A"	7 o	← texte() — un texte gravé
"SALLE B"	7 o	← texte() — un texte gravé

Lettres gravées : ABELS

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerPanneau, Notes

Fonctions du langage appelées :

sprite()	× 1	23 o sur place, dans main()
texte()	× 2	22 o sur place, dans dessinerSalle() + EcrireTexte (10 o)
ecran()	× 2	21 o sur place, dans dessinerSalle() + AttendreVBlank (16 o)
bouton()	× 4	20 o sur place, dans main() + LireManette (37 o)
poser()	× 2	12 o sur place, dans dessinerSalle()
image()	× 1	6 o sur place, dans main() + AttendreImage (28 o)
effacer()	× 1	3 o sur place, dans dessinerSalle() + EffacerFond (17 o)

Tes fonctions appelées :

dessinerSalle() × 3 9 o sur place, dans main()

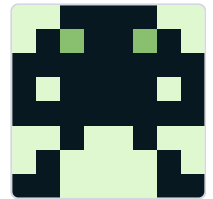
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : deux salles, A et B. Le héros se déplace avec les flèches ;
// en sortant par la droite de la salle A, il arrive dans la salle B
// (collé au bord gauche), et inversement.
//
// Pour changer de salle : ekran(0) (éteindre), effacer() (tout vider),
// redessiner, puis ekran(1) (rallumer) – toujours dans cet ordre.
```

```
#include <Tuile>      // un dessin de 8 × 8 pixels
#include <ecran>      // éteint ou rallume l'écran
#include <effacer>    // efface des cases, ou tout le fond
#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <bouton>     // lit un bouton de la manette
#include <sprite>     // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..####..", ".#-##-#.", "#####", "#.####.#",
    "#####", "..#..#..", ".#....#.", "##....##",
};
```



à l'écran

3. UNE CONSTANTE — GAUCHE_MAX

```
const uint8_t GAUCHE_MAX = 8;      // Le bord gauche, en pixels.
```

4. UNE CONSTANTE — DROITE_MAX

```
const uint8_t DROITE_MAX = 152;    // Le bord droit.
```

5. UNE VARIABLE — SALLE

```
uint8_t salle = 0;                  // 0 = salle A, 1 = salle B.
```

6. UNE FONCTION — DESSINERSALLE()

```
void dessinerSalle() {                // Dessine la salle actuelle.
    ecran(0);                        // Écran éteint : on ne voit pas le changement.
    effacer();                       // Vide tout le fond.

    if (salle == 0) {                // Salle A :
        texte(6, 1, "SALLE A");
        poser(18, 8, 1);            // une tuile A près du bord droit (la sortie).
    } else {                         // Salle B :
        texte(6, 1, "SALLE B");
        poser(1, 8, 1);             // une tuile A près du bord gauche.
    }

    ecran(1);                        // On rallume : la nouvelle salle apparaît d'un coup.
}
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                          // Le jeu commence ici.
    uint8_t x = 76;                  // Le héros, en pixels.
    uint8_t y = 68;

    dessinerSalle();                // La première salle.

    while (true) {                  // Répète pour toujours :
        if (bouton(GAUCHE)) x--;    // Les flèches déplacent le héros.
        if (bouton(DROITE)) x++;
        if (bouton(HAUT)) y--;
        if (bouton(BAS)) y++;

        if (x > DROITE_MAX && salle == 0) { // Sorti à droite de la salle A :
            salle = 1;              // on passe en salle B,
            x = GAUCHE_MAX;         // le héros entre par la gauche,
            dessinerSalle();         // on redessine.
        } else if (x < GAUCHE_MAX && salle == 1) { // Sorti à gauche de la salle B :
            salle = 0;              // retour en salle A,
            x = DROITE_MAX;         // il entre par la droite,
            dessinerSalle();
        }

        sprite(0, x, y, HEROS);    // On pose le héros.
        image();                    // Attend l'image suivante.
    }

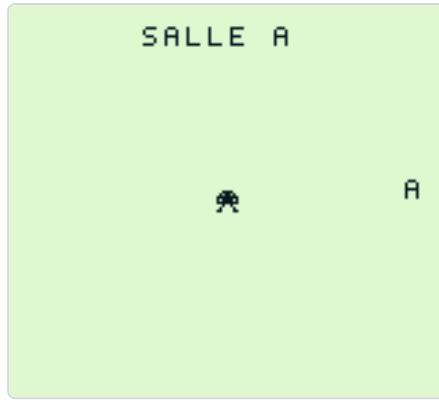
    return 0;                      // Jamais atteint (la boucle ne finit pas).
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

« SALLE A », puis « SALLE B » dès que le héros sort par la droite — il réapparaît collé au bord gauche.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ on démarre dans la salle A
- ✓ sortir à droite fait passer en salle B
- ✓ le héros réapparaît près du bord gauche ($x = 10$)
- ✓ ressortir à gauche ramène en salle A

Ensuite : **134.1. Chapitre 24 — Aller au bout : ce qui vient** — chapitre 24 : Aller au bout. · gameboy3 — écrire une cartouche
Game Boy en C++